

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

2: UML (light) , Classes and Objects

Jan H. Mikkelsen
Cyber og Computerteknologi
2. semester – KBH – F26

AGENDA

- Structured approach to software (system) design
 - Unified Modelling Language (UML) and its use in relation to software design
 - Use-cases and use-case diagrams
 - System definition and modelling
- The class
 - Attributes and methods
 - Examples
- The object
 - Constructor/de-constructor
 - Static and other keywords
- Examples
- Assignments

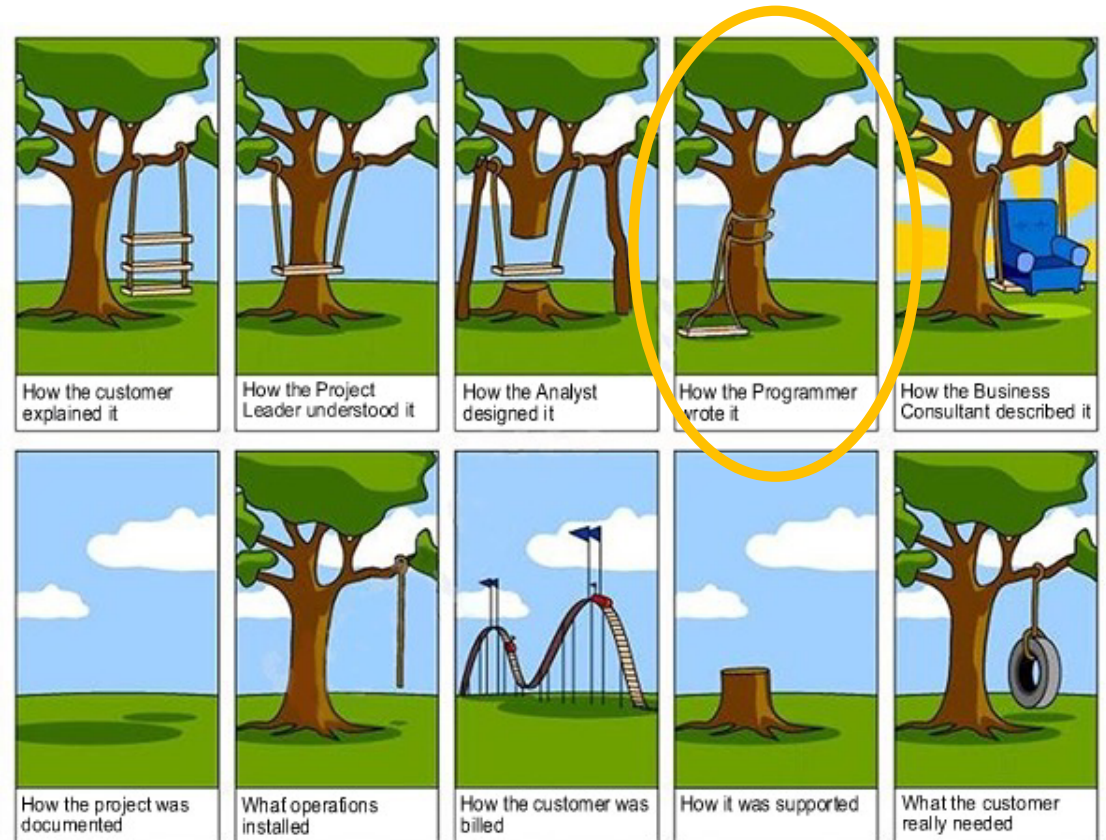
HOW NOT TO DO THINGS

- How to change credit card information for the travel card
 1. Log onto your self-service with username and access code
 2. Select "tank-op-aftale" and stop this
 3. Wait at least 24 hrs. and maximum 7 days. Pass by a train station where you check-in and check-out with a minimum time delay (a null-swipe). This is to register that the "tank-op-aftale" should be stopped.
 4. Wait 24 hrs.
 5. Go back to the website and create a new "tank-op-aftale" on your self-service with your new credit card.
 6. Wait at least 24 hrs. and maximum 7 days. Pass by a train station where you check-in and check-out with a minimum time delay (a null-swipe). This is to register that the "tank-op-aftale" should be stopped.
- This was the actual, official, public procedure (translated into english) on February 6, 2014!!

Kilde: <http://www.rejsekort.dk/brug-rejsekort/saadan-bruger-du-rejsekort/tank-op-aftale/er-dit-visadankort-lukket.aspx> (tilgaaet d. 6 feb. 2014)

OBJECT ORIENTED DESIGN

- We all see and understand a system differently
- This is not very helpful when we write our code
- We need to share the same view of our:
 - System
 - Models
 - Interfaces
 - Functionality
- If we misunderstand it, we will face consequences of some sorts



OBJECT ORIENTED DESIGN

4

- The background for OO design is a firm understanding of the context our code should interact in
 - We need first to fully understand our system context
- Can you create the code for our travelling system?
- How would you go about this challenge?
- We need a unified (and agreed upon) way of describing functionalities



UNIFIED MODELLING LANGUAGE (UML)

- UML is – as the name suggests – a full modelling language frame-work used to describe (software) systems
- UML contains many different diagram types, such as:
 - Class diagrams
 - Use-case diagrams
 - Sequence diagrams
 - Activity diagrams
 - etc.
- They are all different ways to create an unambiguously and shared picture of what we are trying to design

Actors

Usage, involvement

Classes

Association, inheritance, aggregation

Objects

Messages, lifelines

States

Transitions, behaviors

Activities

Guards, synchronization

Packages

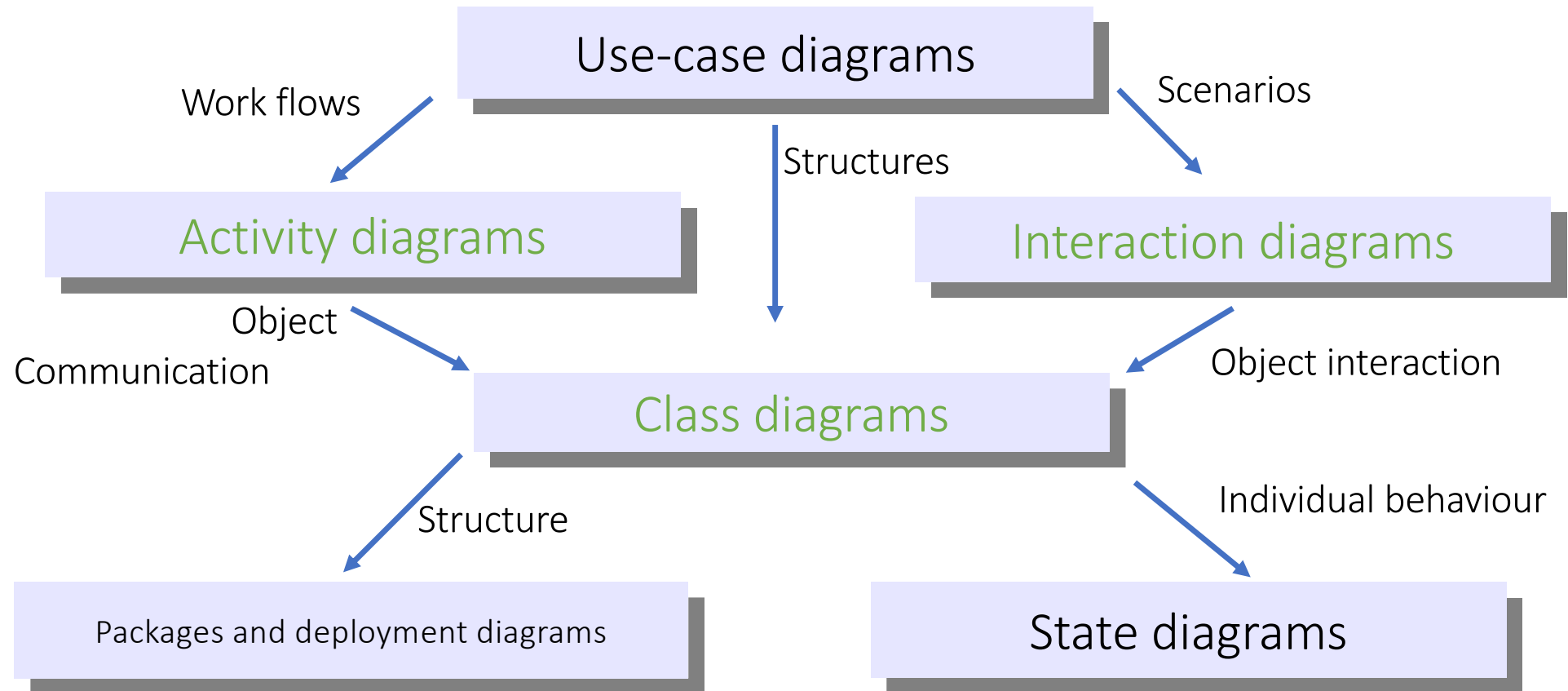
Software dependencies

Processors and Nodes

Component dependency

UNIFIED MODELLING LANGUAGE (UML)

6



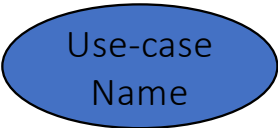
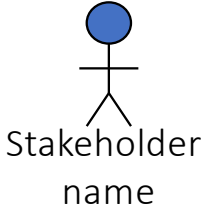
UNIFIED MODELLING LANGUAGE (UML)

7

- UML specifies use-cases as a methodology to understand requirements
- A use-case diagram captures:
 - A complete set of functionality of a complete system which has a value to a user
 - Stakeholders that are external to a system.
This can be a person, hardware, other systems, or something else
 - Stakeholders are characterized by their role and not by who they are
 - The system in this context can be viewed as a black box
 - Does not include design – that is what we need to focus on
 - Comes with only the amount of use cases that are necessary to understand required functionality
- This is very useful information for designing and constructing our code

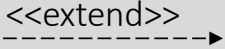
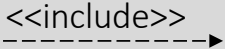
USE-CASE DIAGRAMS

- Use-case diagrams contain a few basic components

Component	Description	Pictogram
Use case	A sequence of actions that the system can perform in interaction with the external stakeholders	
Stakeholder	A descriptive way to formalize an entity that interacts with the system.	
System boundary	A representation of the limits/ boundaries of the system.	

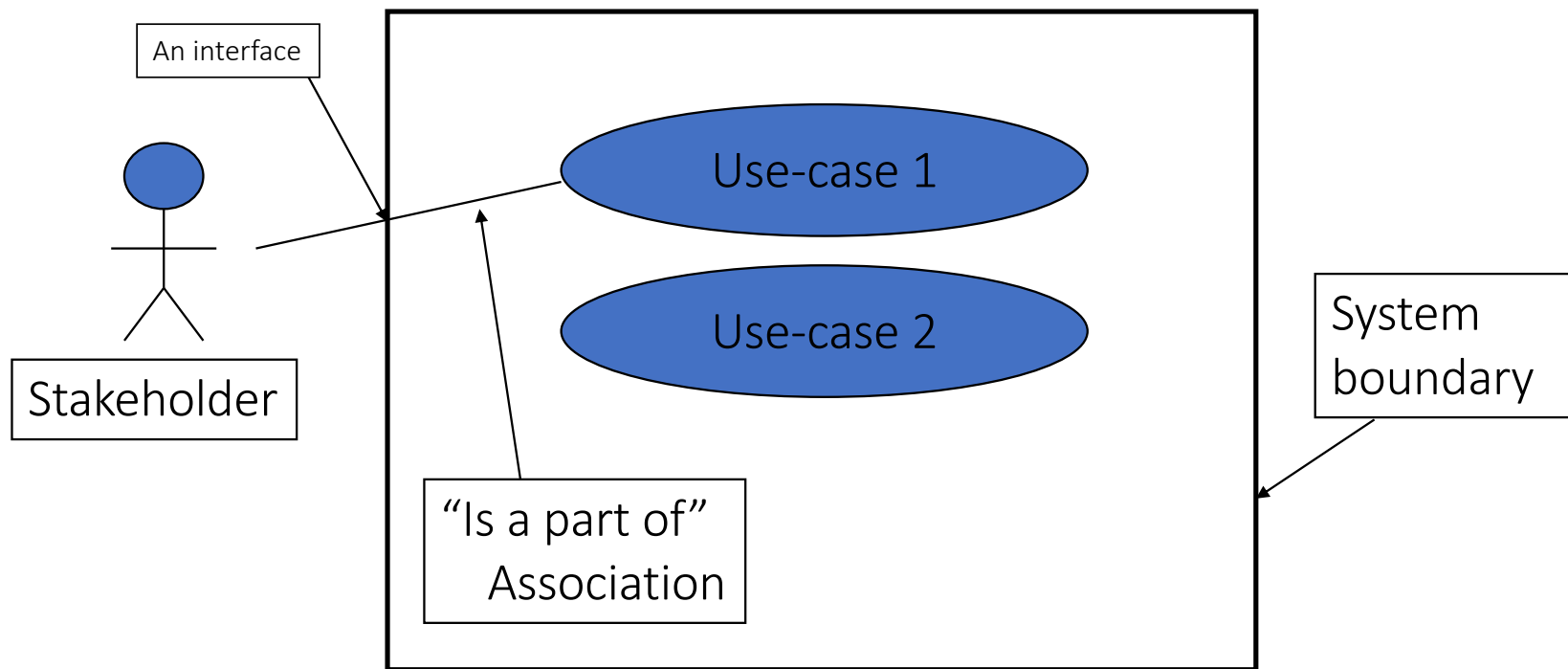
USE-CASE DIAGRAMS

- In addition, the use-case diagram specifies the nature of relations

Relation	Description	Diagram
Association	Interaction between a stakeholder and a use-case	
Generalization	Relation between a generic use-case and a specific use-case	
Extension	Relation between an extended use-case based on an existing use case	
Include	Relation between a use-case and one which includes the specific use-case	

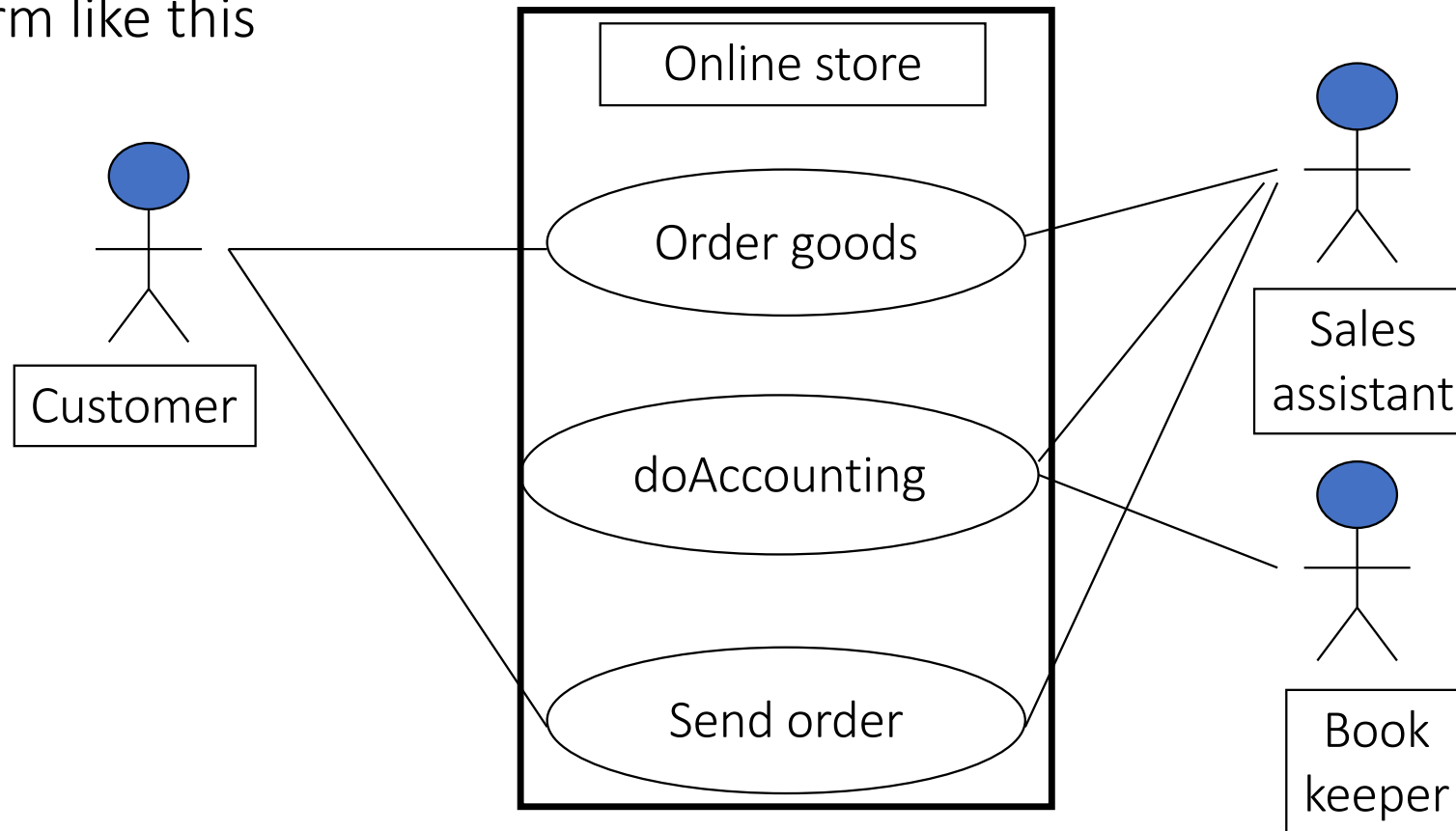
USE-CASE DIAGRAMS

- The use-case diagram is a graphical representation that provides an overview of the entire system that we are trying to design



USE-CASE DIAGRAMS

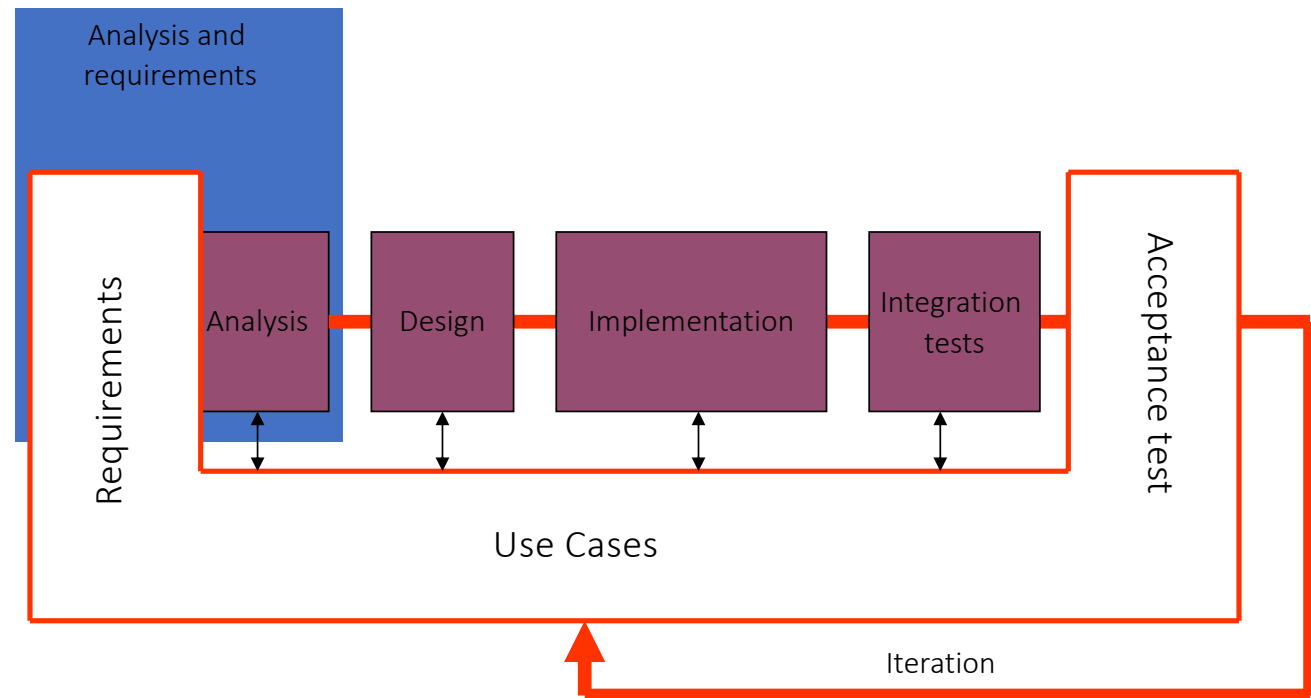
- As an example, the use-case diagram for an online store could take on a form like this



USE-CASE FOCUS – REVISITING OBJECTIVES

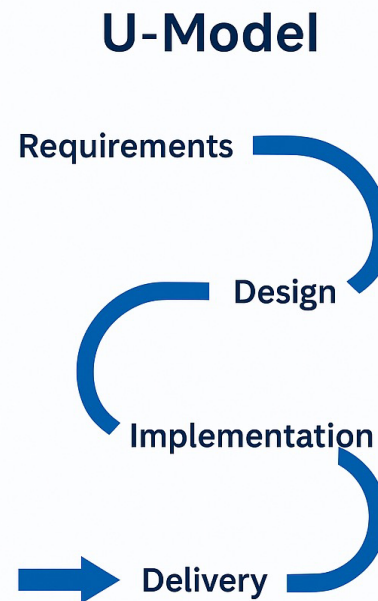
12

- Use-cases are critical to understand the requirements that we are facing
- Various approaches can be applied during the design phase, here illustrated using the U-model
- Other options include:
 - V-model
 - Spiral model



USE-CASE FOCUS – REVISITING OBJECTIVES

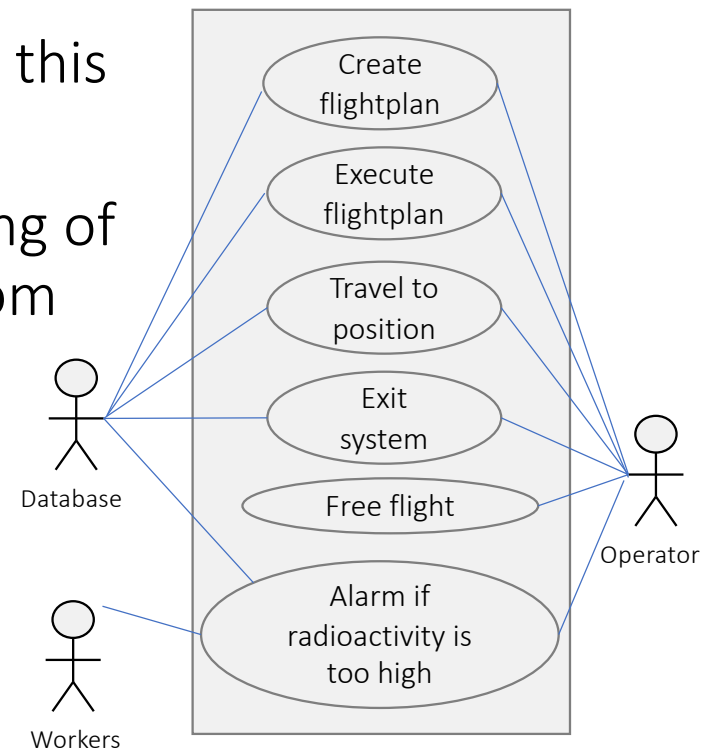
- There are pros and cons ... some approaches are better for implementation phases while others are better for overall product development phases
- If we aim to solve a problem then we need to revisit this every now and then
- We need to make sure that the problem is still there, that requirements have not changed etc.
- Different models for different purposes



EXAMPLE – DEVELOPMENT OF DRONE SYSTEM

14

- A use-case diagram of a system with a drone to assess level of radioactivity in any given area
- Initial design considerations has resulted in this use-case diagram
- On such basis we have a good understanding of what our system needs to do and with whom we are interacting
- The next step is to turn this information into code, that is related to our initial design and that solves the problem that we set out to solve in the first place



EXAMPLE – DEVELOPMENT OF DRONE SYSTEM

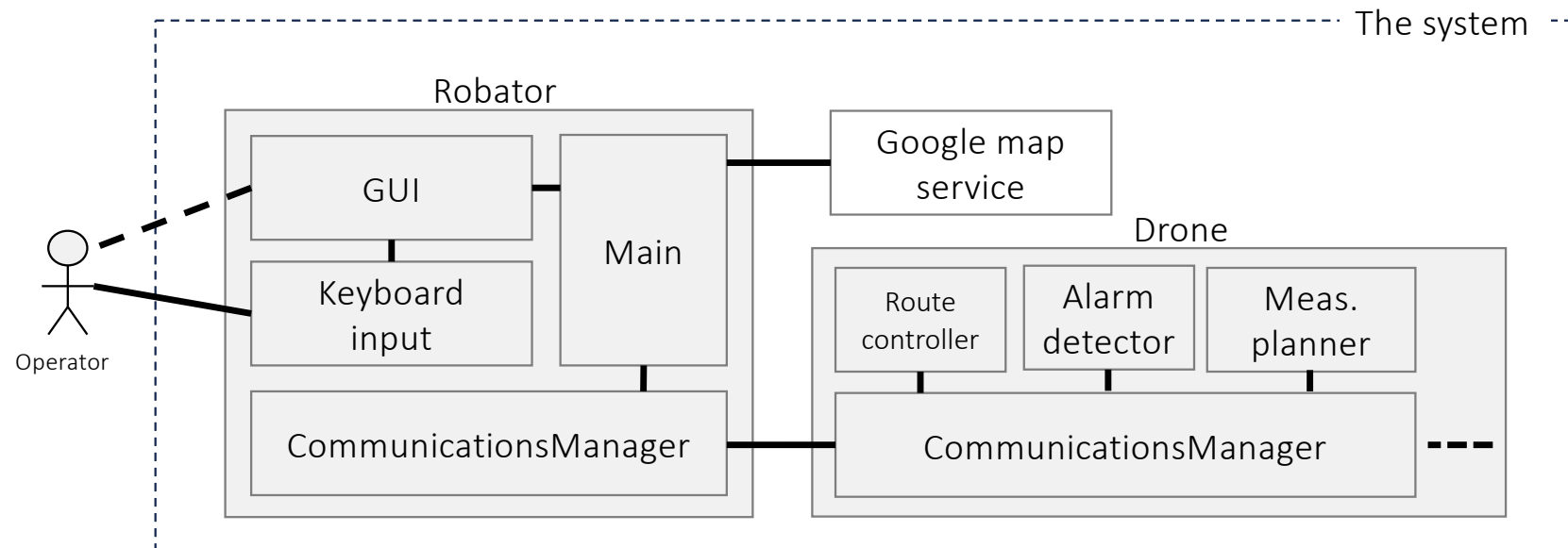
- Description of use-cases
 - 1.Operator turns on his program (let's call it Robator) – a part of the system
 - 2.Robator connects to our drone
 - 3.The operator inserts coordinates into Robator that the drone should visit
 - 4.The operator clicks OK to a suggested plan by Robator (alternatively, makes some changes)
 - 5.Robator sends the flight plan to the drone
 - 6.The drone accepts it with an OK, which Robator shows to the operator
- Some alternatives
 - The route can get too long compared to the life-time of the battery, which the system should inform the operator about
 - Communication may be poor during the flight, which also should be informed to the operator, etc.

EXAMPLE – DEVELOPMENT OF DRONE SYSTEM

- From the set of use-cases we can derive requirements
- Not covered in this course, but some example requirements can be
 - Information must be shown to the operator
 - Robotor must be able to communicate with the drone
 - Robotor must be able to accept input from operator
 - The drone must be able to navigate in the field
 - The drone must be able to communicate with Robotor
- Disclaimer: The above requirements are not super-duper, but just indicative that a part of this analysis are requirements
- From the requirements we may be able to derive a design of a system

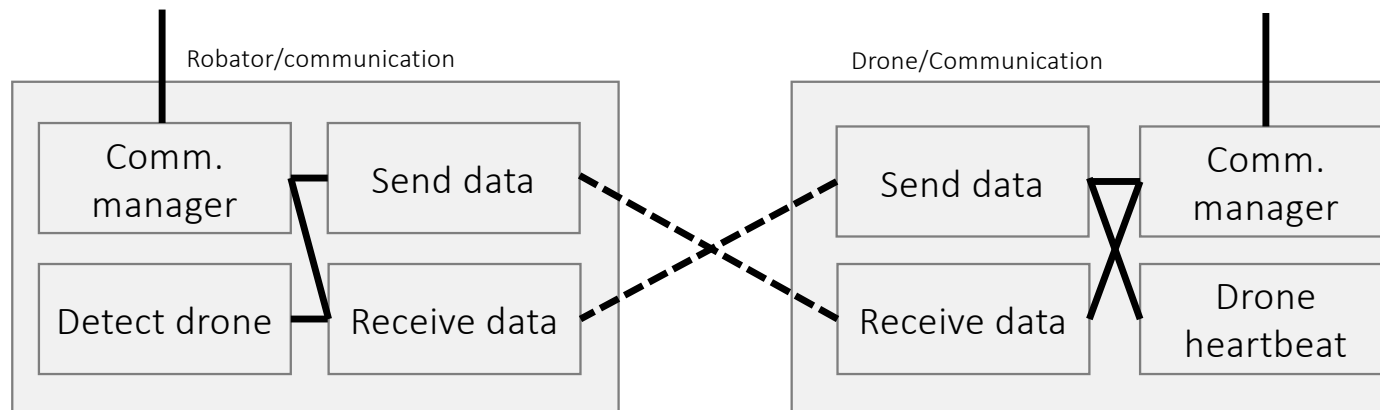
EXAMPLE – DEVELOPMENT OF DRONE SYSTEM

- Again, note that this is not a full-fledged example system diagram, but rather it serves to illustrate the steps needed
- With this kind of systems understanding we are getting closer to our implementation – we recognise “software modules”



EXAMPLE – DEVELOPMENT OF DRONE SYSTEM

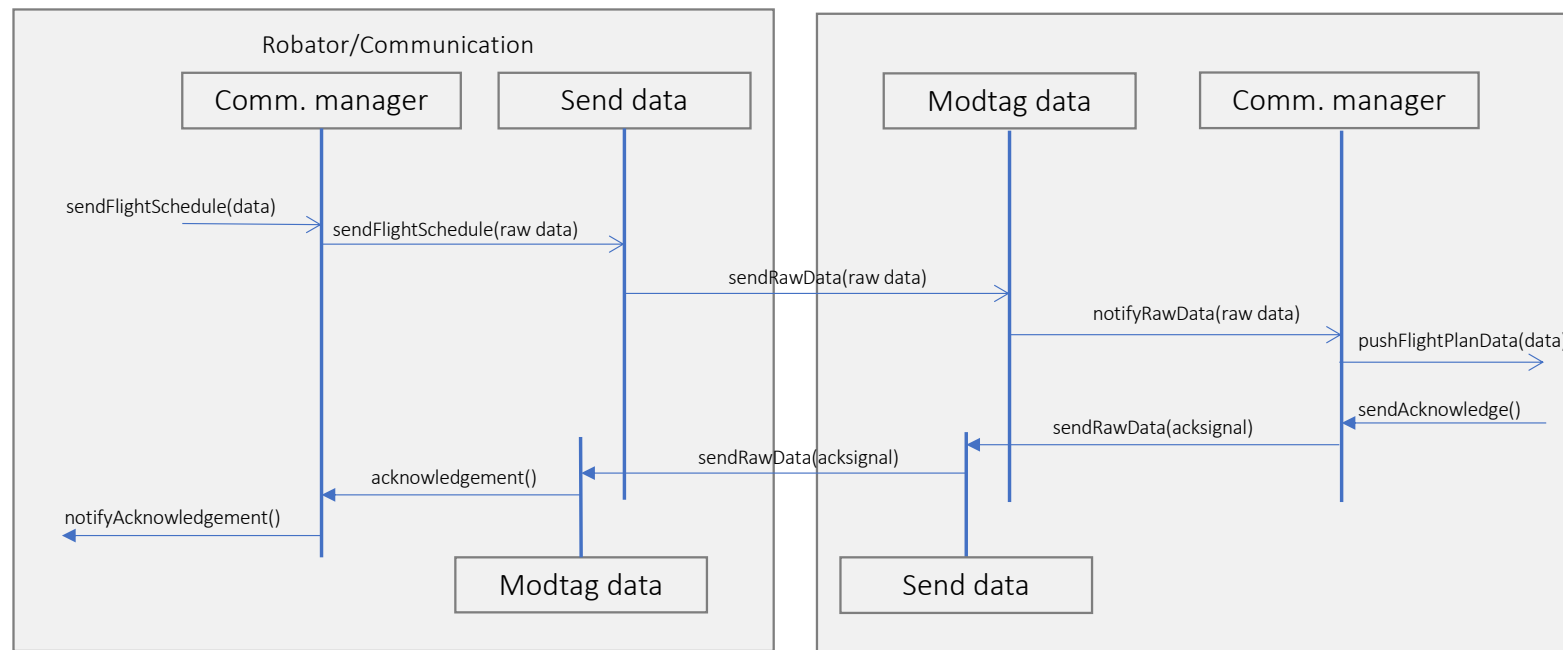
- Each module may subsequently be split into sub-modules



- For each (sub) module we may be able to identify one or possibly more ways to use the module
 - Initialize(), SendData(), ReceiveData(), SleepModule(), etc.

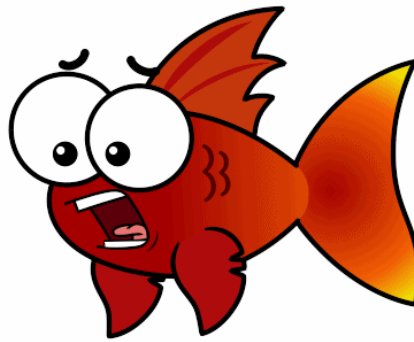
EXAMPLE – DEVELOPMENT OF DRONE SYSTEM

- We are practically already in the midst of the design phase now – drawing your (sub) modules and their interactions with each other using a timewise approach helps to further specify the task



BREAK – PAUSE(FISK)

20



CLASSES & OBJECTS – KITCHEN STYLE

21

- We will start with an analogy, which (hopefully) makes more sense as we progress
- A cookie cutter defines a general description for some form of cookie
- Many cookies can share this form
- But the cutter is not itself a cookie
 - In this analogy the cutter resembles a class



CLASSES & OBJECTS – KITCHEN STYLE

22

- To make a cookie, we need to follow a predefined procedure, and we need to use some ingredients
- The method for creating an object is called a constructor
- Constructors can have defined input parameters
- Constructors are special methods that belong to the class



CLASSES & OBJECTS – KITCHEN STYLE

23



- Our cookies are similar in that they are all cookies, but they are still distinct entities, regardless of how much or little they vary
- Each individual cookie is an instance of a class– an object
- Each object has its own attributes
- Even if the attributes of two objects are identical, they are not equal
- Objects that are related to the same class share a type

CLASSES & OBJECTS – KITCHEN STYLE

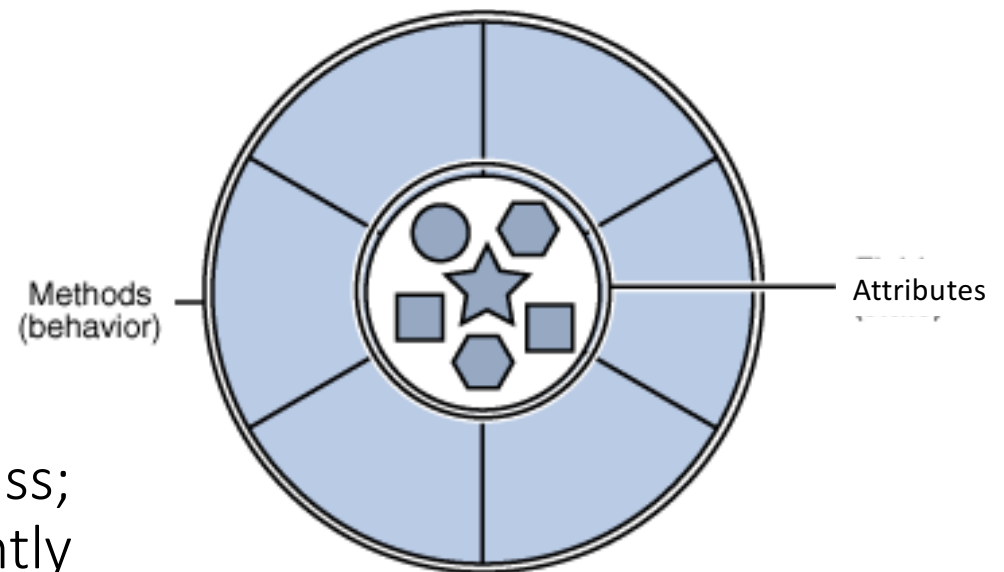
24



- Things we can do with a cookie, like decorating, can be describes as something we can do to any cookie, or as a feature of a specific cookie
- We can write methods that belong to a class, which takes an object as an input
 - `Decorate(myCookie);`
- We can write methods that belong to an object, and are run in the context of said object
 - `myCookie.Decorate();`

CLASSES – WHAT IT THIS?

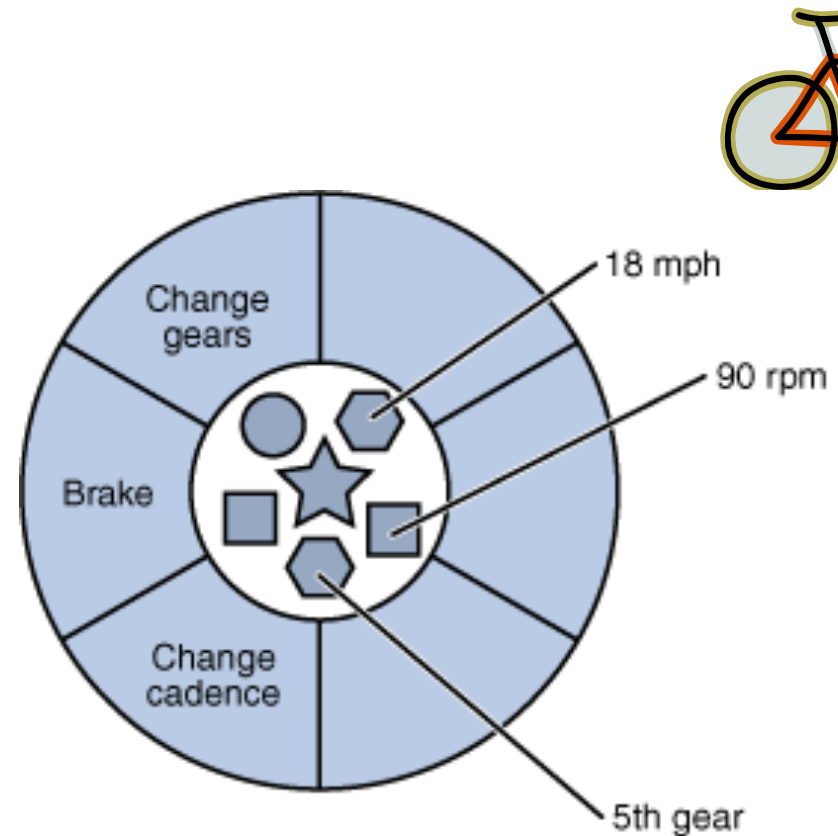
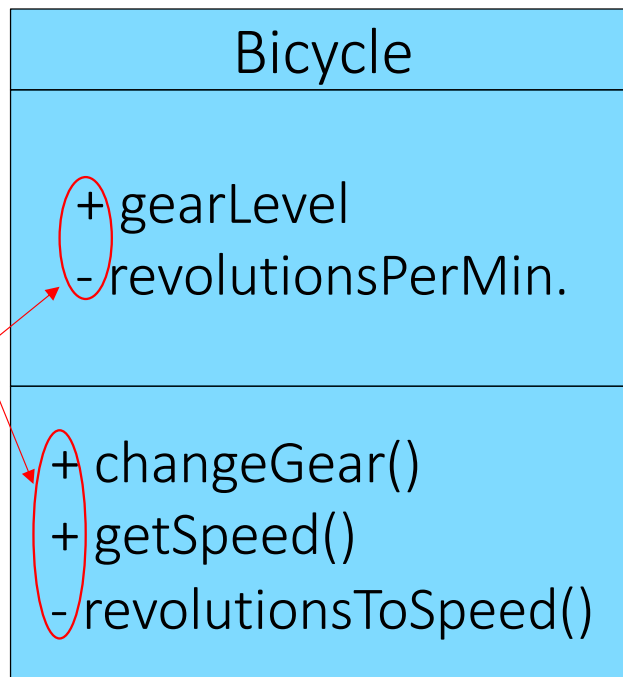
- Classes are *definitions* of groups of objects with the same attributes and operations
- They act as “blueprints” for the actual objects
- Individual objects of a given class are called *instances* of that class; they can be created and subsequently destroyed
- In some (rare) cases, objects of a class cannot be created; such classes are called *abstract*



CLASSES – WHAT IT THIS?

- The bicycle is a classical example

UML visibility/accessibility indicators



CLASSES – WHAT IT THIS?

- UML visibility symbols – indication of who is allowed to access the specific method or attribute

Symbol	Meaning	Java Equivalent	Who Can Access It?
+	Public	public	Any class anywhere
-	Private	private	Only the same class
#	Protected	protected	Same class, subclasses same package
~	Package / default	<i>default</i> (no keyword)	Any class in the same package

DATA ENCAPSULATION

- When creating our classes it is important to pay careful attention to the “openness” of the code
- Something may be essential to share to ‘everyone’ while other things must be kept ‘secret’ ... kept within the class

Bicycle
+ numberOfGears + colour - Wheels - Tire
+ Bicycle() + getFrontWheel() + getBackWheel() + changeGear() - moveChainToSprocket()

Class name

Public attributes

Private attributes

Constructor

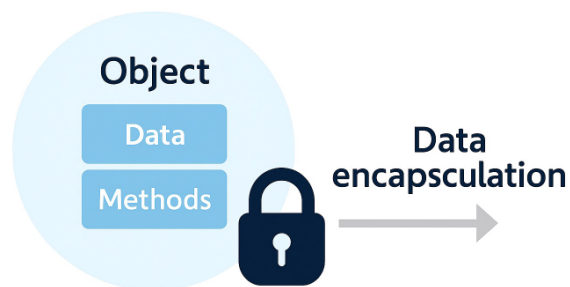
Public operations

Private operations

DATA ENCAPSULATION

29

- Data encapsulation refers to two distinct but related concepts
- Bundling data with the methods that operate on the data
- Restricting access to some object components, which we refer to as information hiding



Bicycle	Class name
+ numberOfGears	Public attributes
+ colour	Public attributes
- Wheels	Private attributes
- Tire	Private attributes
+ Bicycle()	Constructor
+ getFrontWheel()	Public operations
+ getBackWheel()	Public operations
+ changeGear()	Public operations
- moveChainToSprocket()	Private operations

DATA ENCAPSULATION

30

- Bundling data and related methods
- Reduces problems stemming from external code, that would have to be concerned with the inner workings of an object
- Helps in providing consistent interfaces
- Helps keep things organized
- Encourages decoupling

Bicycle	Class name
+ numberOfGears	Public attributes
+ colour	
- Wheels	Private attributes
- Tire	
+ Bicycle()	Constructor
+ getFrontWheel()	Public operations
+ getBackWheel()	
+ changeGear()	Private operations
- moveChainToSprocket()	

DATA ENCAPSULATION

31

- Information hiding
- Supports the integrity of objects and hinders users from manipulating parts that could lead to an invalid, inconsistent or unstable state
- Makes it explicit what a user is allowed to do, which can make it less complex
- Separates the concept of what something does from how it is implemented

Bicycle	Class name
+ numberOfGears	Public attributes
+ colour	
- Wheels	Private attributes
- Tire	
+ Bicycle()	Constructor
+ getFrontWheel()	Public operations
+ getBackWheel()	
+ changeGear()	Private operations
- moveChainToSprocket()	

CLASSES – WHAT IT THIS?

- Other examples of classes with indication of possible attributes and methods



Waste pump
- rotational speed - lifting height - power consumption
+ reset() + setSpeed(rotPerMin) + getSpeed() + start() - number of running hours - max flow - max current, ...

Objects

Drone
- rotational speed - horizontalSpeed - powerConsumption
+ reset() + setVertSpeed() + start(),stop() - battery info - battery power - number of running hours,

Objects



CLASSES

- Recall that in OO the class is the basis of all our code work
- In Java, each class is often coded in a separate file

```
public class HelloWorld {  
    public static void main(String[] args) {  
        //This is a comment  
        System.out.println("Hello, World!");  
    }  
}
```

Filename: HelloWorld.java

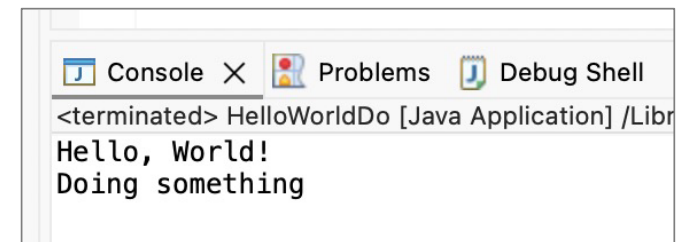
- We can use this class actively, from somewhere else in our code by adding something like this – object creation

```
HelloWorld helloWorld = new HelloWorld();
```

CLASSES

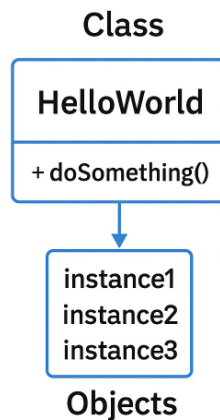
- We can add methods (functions) to our classes and then actively use these for our various purposes
- The strength of this is that we can model our world into code

```
public class HelloWorld {  
    public static void main(String[] args) {  
        //This is a comment  
        System.out.println("Hello, World!");  
        HelloWorld helloworld = new HelloWorld();  
        helloworld.doSomething();  
    }  
    public void doSomething() {  
        System.out.println("Doing something");  
    }  
}
```



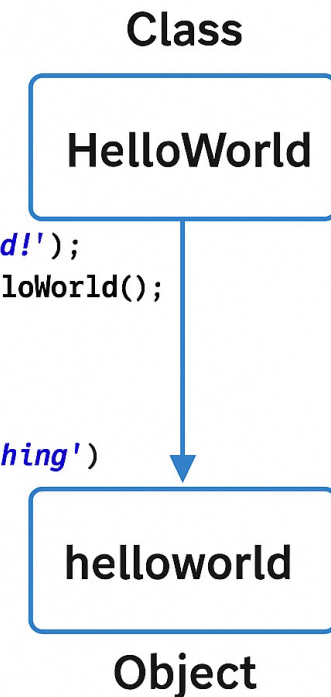
CLASSES

- With this code we have the blueprint as the Class and from this we create a new object, helloWorld, that is an instance of that blueprint
- The arrow shows that the 'new HelloWorld()' creates an object from the class



```

public class HelloWorld {
    public static void main(Srings) {
        //This is a comment
        System.out.println('Hello, World!');
        HelloWorld.helloworld = new HelloWorld();
        helloworld.doSomething();
    }
    public void doSomething() {
        System.out.println('Doing something')
    }
}
  
```



CODE EXAMPLE OF CLASS

- Recall that the class is the blueprint ... here again using the bicycle as example
- Contains an error
 - Speed needs updating when gear is changed
 - If added to changeGear the formula would be repeated
 - Private method would maybe be better

A method to get an attribute

```

3 public class Bicycle {
4     private int cadence = 0;
5     private float speed = 0;
6     private int gear = 1;
7     private float tireSize = 0;
8     private float tireDiameter = 0;
9
10    //Constructor
11    public Bicycle(float ts, float td) {
12        tireSize = ts;
13        tireDiameter = td;
14    }
15
16    //Methods
17    public void changeCadence(int newValue) {
18        cadence = newValue;
19        speed = (float) (Math.PI*(tireDiameter + (2*tireSize))*gear*cadence);
20    }
21
22    public void changeGear(int newValue) {
23        gear = newValue;
24    }
25
26    public void applyBrake(int decrement) {
27        speed = speed - decrement;
28    }
29
30    public float getSpeed() {
31        return speed;
32    }
33
34    public void printBicycleState() {
35        System.out.println("Cadence: "+cadence+", speed: "+speed+", gear: "+gear);
36    }
37
38 }

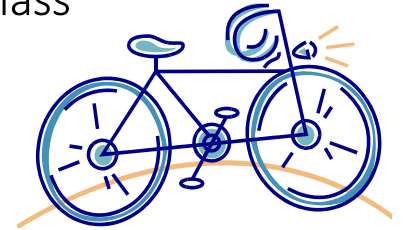
```

This is a class

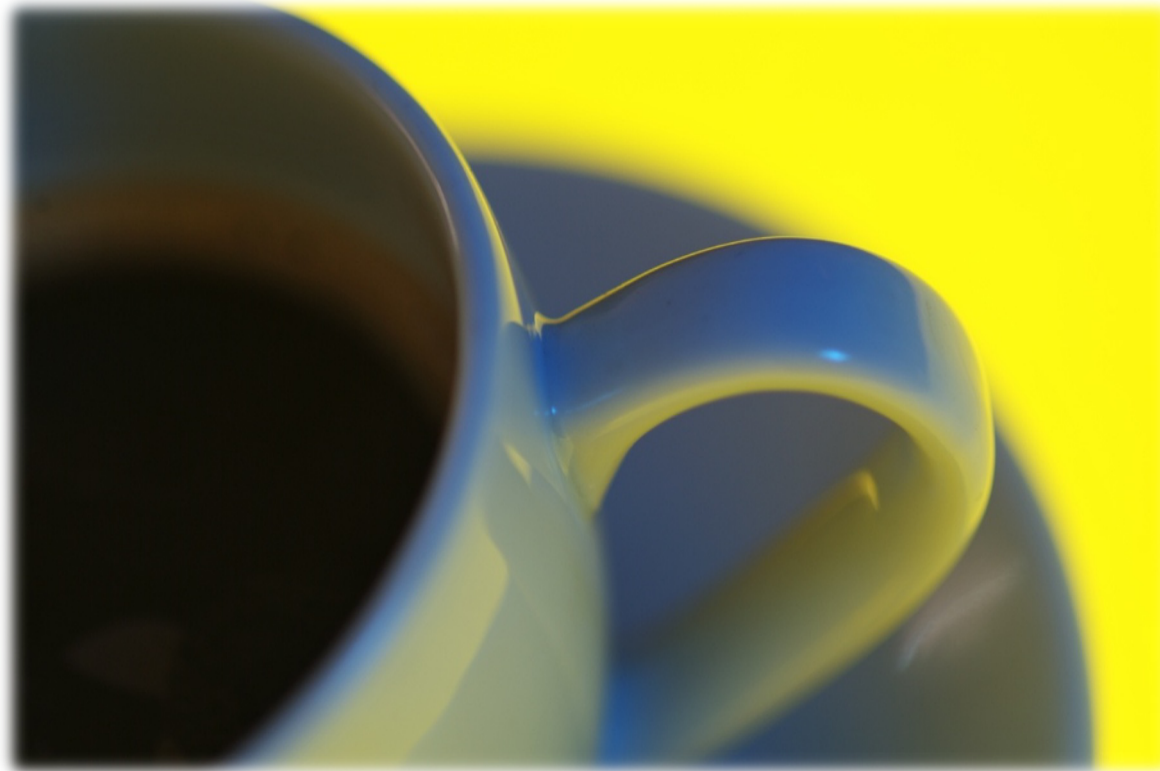
An attribute

A local variable

A method to change an attribute

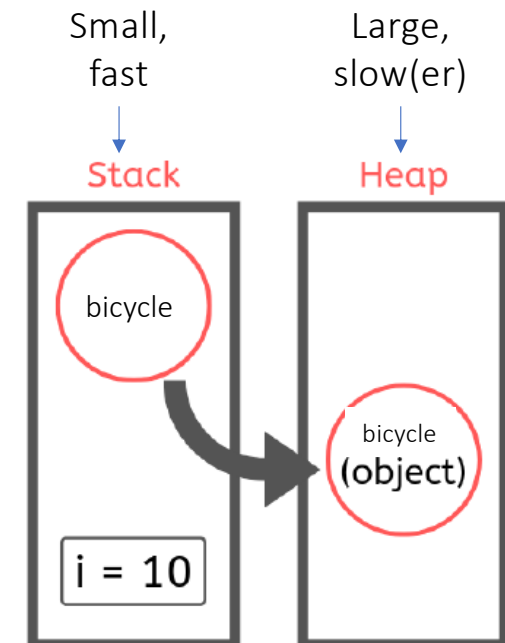
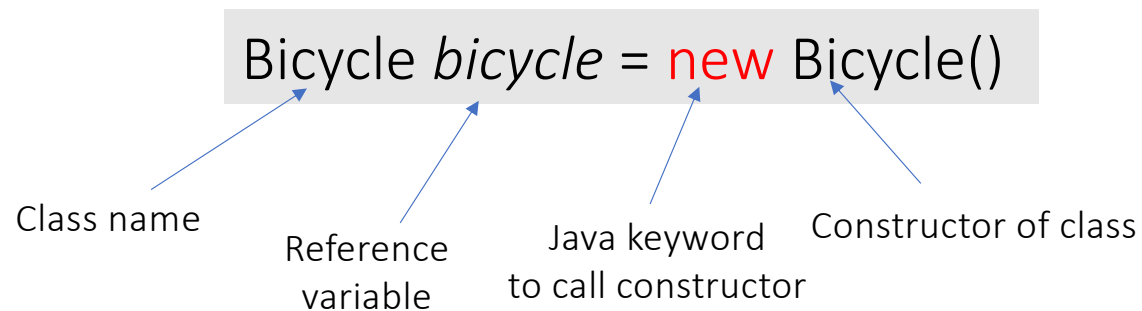


"BIKER BREAK!"



REFERENCE VARIABLE FOR OBJECTS

- Objects are accessed via a reference variable
- The reference variable is a pointer to the location in memory where the object, once created, is stored
- In Java a new object is created by



USING OBJECTS AND CLASSES

Class (the blueprint)

Bicycle



bike1



bike2



Instances
(we can work with those)

```

3 public class Bicycles {
4
5     public static void main(String[] args) {
6
7         Bicycle bike1 = new Bicycle(23, 600);
8         Bicycle bike2 = new Bicycle(25, 700);
9
10        System.out.println("Bicycle 1:");
11        bike1.changeCadence(50);
12        bike1.printBicycleState();
13        bike1.changeGear(2);
14        bike1.printBicycleState();
15
16        System.out.println("Bicycle 2:");
17        bike2.changeCadence(50);
18        bike2.printBicycleState();
19        bike2.changeGear(2);
20        bike2.printBicycleState();
21
22        System.out.println("Bicycle 1:");
23        bike1.applyBrake(10);
24        bike1.printBicycleState();
25        bike1.changeGear(1);
26        bike1.changeCadence(20);
27        bike1.printBicycleState();
28
29    }
30

```

We create a couple
of instances of
bicycles

Here we use our object
as intended

INPUTS AND OUTPUTS

- A class shares similarities with modules
 - It may hold (several) input and output methods
 - It may hold (several) functionalities
 - There is an intuitive relation between functionalities
- Input and output can also be object references
 - A Person class can use a Bicycle, e.g. *person.use(bicycle)*
 - A BicycleShop can return a Bicycle, e.g. *Bicycle myBike = bicycleShop.buy(money)*
- *It is only the references to the object that is exchanged!!*
 - This can have some implications once your code involves concurrent (multi-threaded) activities

If two people share the **same bicycle**:

- they can change each other's seat height
- move it to a different location
- change the gears
- use it at the same time → conflict

But if each person has a **copy of the bicycle**, no conflicts occur.

ENCAPSULATION - AGAIN

41

- Encapsulation is often very useful
- In code (1) there is much less code than in (2), so why is (2) better?
- Consider the bicycle class
 - `bicycle.colour = 4;`
 - `bicycle.changeColour(4);`
- Now I don not have to recall how to translate from cadence to speed as the code is already there

1

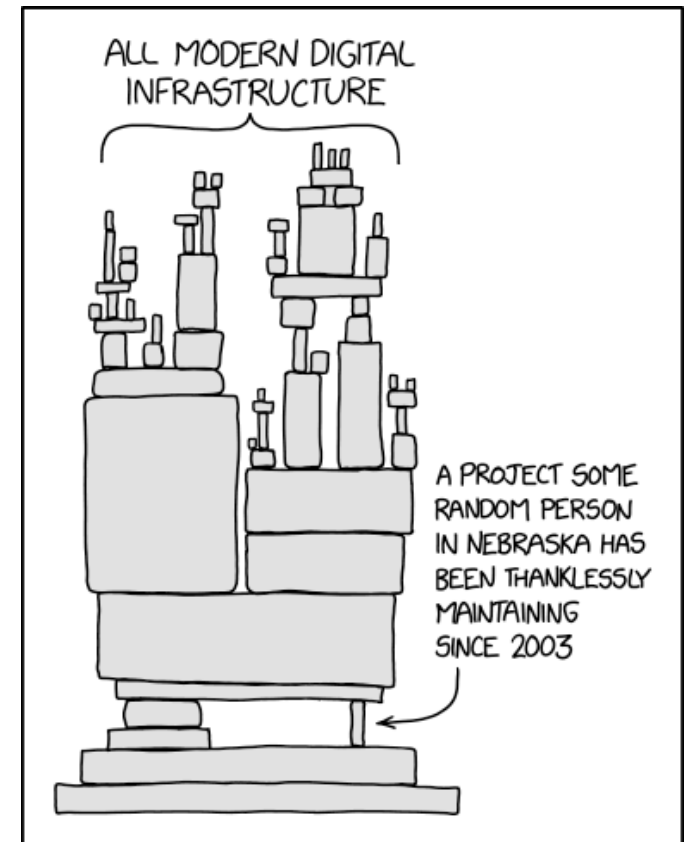
```
class Bicycle {  
    float speed = 0;  
    public Bicycle(){  
    }  
}
```

2

```
class Bicycle {  
    private float speed = 0;  
    public Bicycle(){  
    }  
    public void changeCadence(int rpm){  
        this.cadence = rpm;  
    }  
    public float getSpeed(){  
        return cadence*gear;  
    }  
}
```

ENCAPSULATION - AGAIN

- But hey, is $\text{speed} = \text{cadence} * \text{gear}$ not a bit simplistic?
- Indeed, and it is easy to make such mistakes along the way
 - Bug, mistakes, misunderstandings, "I had to leave early last Friday",
 - We needed some equation with tire information and π was also somehow involved
- If we have protected our state variables, we can easily improve and update functionality
 - Otherwise, all places where code uses speed needs to be updated



THIS – WHAT IS ‘THIS’?

- ***This*** is a keyword that allows us to specify “ourselves” as the attribute object owner
- Consider case (1) and (2)
 - In (1) we clearly refer to “our own” object and its attribute cadence
 - > We set Bicycle cadence to value cadence
 - In 2) we refer to the local input variable – which makes little sense.
 - > cadence = cadence and so what?

1

```
class Bicycle {  
    private float cadence = 0;  
  
    public void changeCadence(float cadence){  
        this.cadence = cadence;  
    }  
}
```

2

```
class Bicycle {  
    private float cadence = 0;  
  
    public void changeColour(float cadence){  
        cadence = cadence;  
    }  
}
```

VISIBILITY MODIFIERS

- Words put in front of methods and attributes (but also class'es)

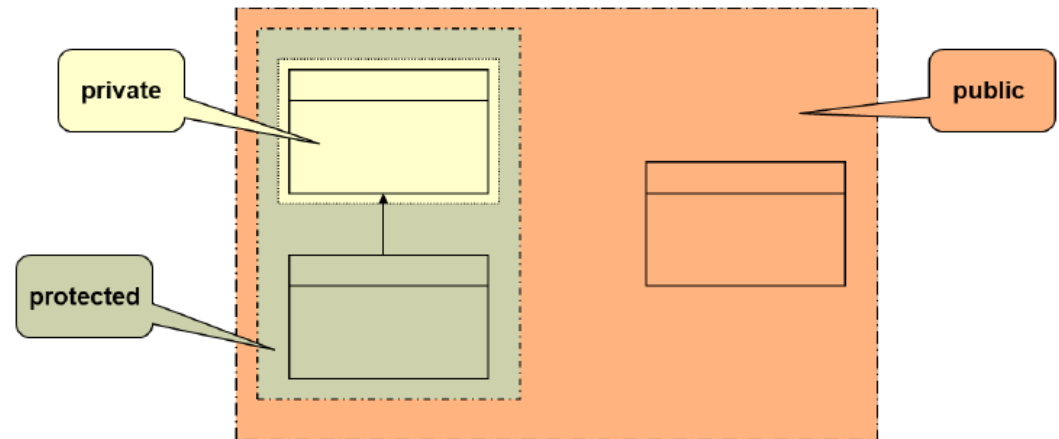
Private: We keep methods and attributes to ourselves.
Methods and attributes are not visible outside the class.

Protected: We keep methods and attributes to our family.
Methods and attributes are only visible to children (via inheritance)

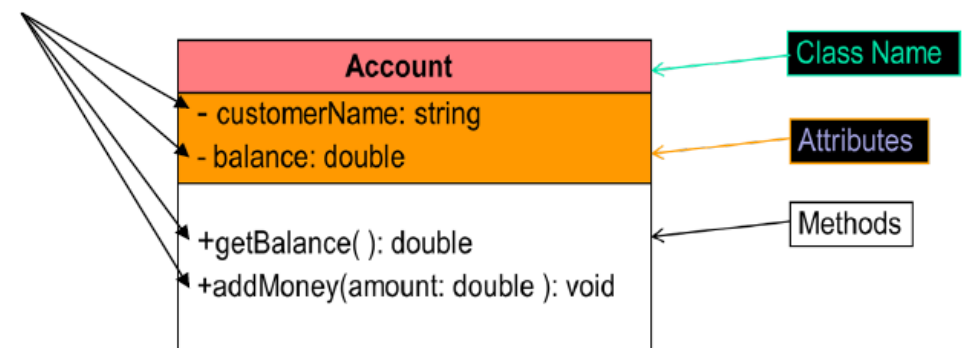
Public: We are open to all outsiders.
Methods and attributes can be accessed to all who knows the class

VISIBILITY MODIFIERS

- The notion for visibility
 - Public: **+** method/attribute
 - Protected: **#** method/attribute
 - Private: **-** method/attribute
- Very useful when we go deeper into OO design
- Default is public – so, if you don't actively hide your privates then they are public!!



Visibility modifiers



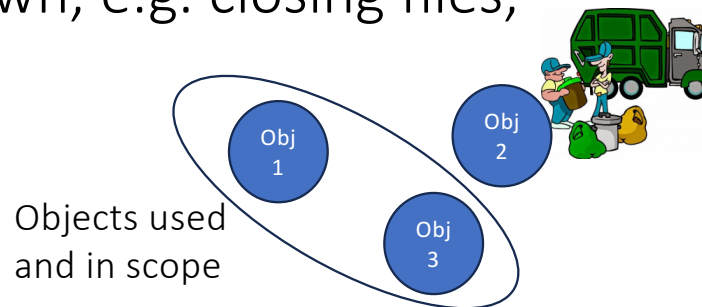
CONSTRUCTOR AND DESTRUCTOR

- The constructor is called when we initiate the object
 - `Bicycle myBicycle = new Bicycle();`
 - Constructor is a method without any return value (it is implicit an object of that class)
 - This is a deterministic process!!
- Destroying an object (cleanup) is NOT a deterministic project in Java as it depends on many different things
- `finalize()` was meant to run when an object dies but it was deprecated in Java 9 and completely removed in Java 18
- Objects using external resources must clean up explicitly using `AutoClosable()` – more about this later in the course

```
class Bicycle {  
    private int cadence;  
    public Bicycle(){  
        this.cadence = 0;  
    }  
}
```

GARBAGE COLLECTION AND DECONSTRUCTION

- Behind the scene, Java is keeping track of objects and how they are referenced
- If nothing is referenced to an object, it means that it is not used anymore
 - It is cleared from memory at some point in time
- Deconstructor can be made, but is a bit more complex – java mostly rely on garbage collection and after that, resources are gracefully shut down, e.g. closing files, freeing ports, ...



```
If (numOfBicycles>0){  
    Bicycle localCycle = new Bicycle();  
    ....  
}
```

localCycle is no longer valid – so it is removed

```
Bicycle mainBicycle;  
If (numOfBicycles>0){  
    Bicycle localCycle = new Bicycle();  
    ....  
    mainBicycle = localCycle;  
}
```

localCycle now lives on as mainBicycle

THE 'NULL' VALUE

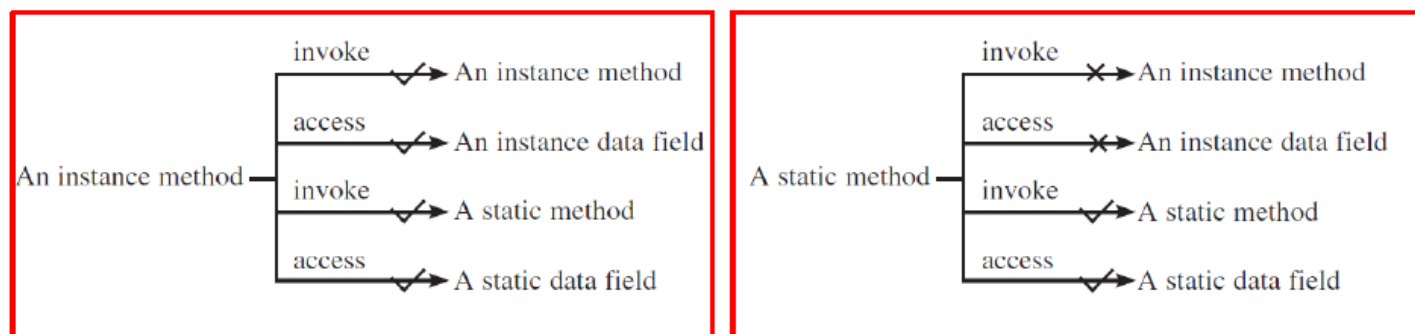
- Usually, one creates an object and assigns it to a variable
 - `Bicycle bicycle = new Bicycle();`
- Sometimes it is convenient to create the object later, and just declare the class type
 - `Bicycle bicycle;`
- Until the constructor is called via `new Bicycle()`, then `bicycle` holds the value `null` – which isn't really a value, but a reference to address 0 is not valid, and is essentially saying that `bicycle` has not been created
 - A highly common error is the dreaded *null pointer exception* meaning that some object was not created somewhere in the code – you have forgotten to `new Bicycle()`
 - Null pointer exceptions often happens other places than where the error is
- Setting a variable to `null` also basically tells the garbage collector that the object is no longer needed, e.g. **`bicycle = null;`** will effectively destroy the `bicycle`

STATIC METHODS

- What is it?
 - We use the keyword ***static*** in front of the method name
- Why do we need it?
 - In some cases, we cannot, or do not want to, have an object instance – we just want to use a method
 - A **static method** is a method that belongs to the **class itself**, not to an object instance
 - A typical example is ***main(String[] args)***, which is commonly the first method to be called – and at that stage we do not have any code to setup an instance. So having main as a static function solves a chicken-egg problem
 - An instance of the Math library does not make much sense by design, but we still need to use Cos() and Sin(), every now and then, so we need to use ***Math.cos()*** and ***Math.sin()***
- When would I need it?
 - Whenever, OO does not add value – where creating an object makes no sense

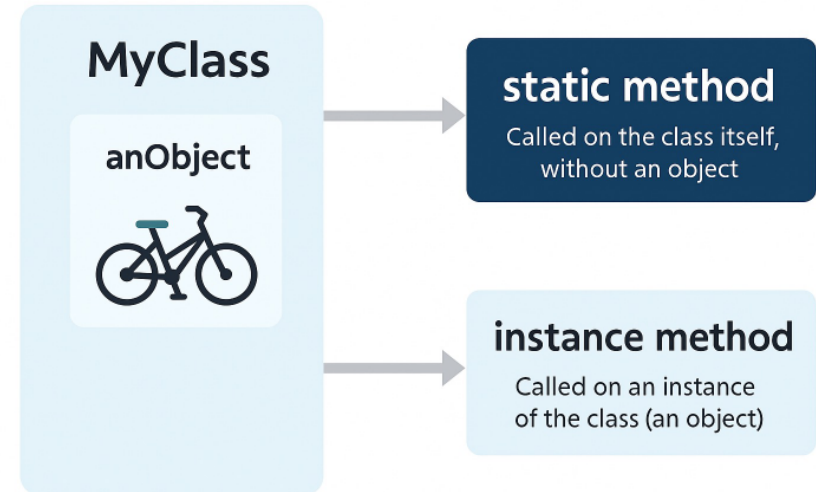
STATIC METHODS

- When to use static methods
 - Static methods do come with limitations, so it is often a design choice
 - Global constants may effectively be implemented as static attributes
 - Methods that you want to use, but don't want to go through the process of instantiate an object – this kinds of violate the OO design principles, but is sometimes very useful
- This is a very deliberate implementation in Java to step outside the OO model when object-oriented-ness is not useful. But it is not a “hack”; it is a core, intentional language feature that:



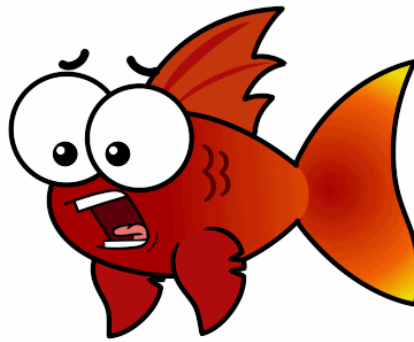
STATIC METHODS

- A rule of thumb to follow, for when to make use of static methods, is to use a static method when:
 - It does *not* (and never will) need object state
 - It represents a general operation and is not tied to any one object
 - Creating an object would be meaningless (e.g., Math)
 - You need a function available before objects exist (main)
- Otherwise, one should always use normal instance methods



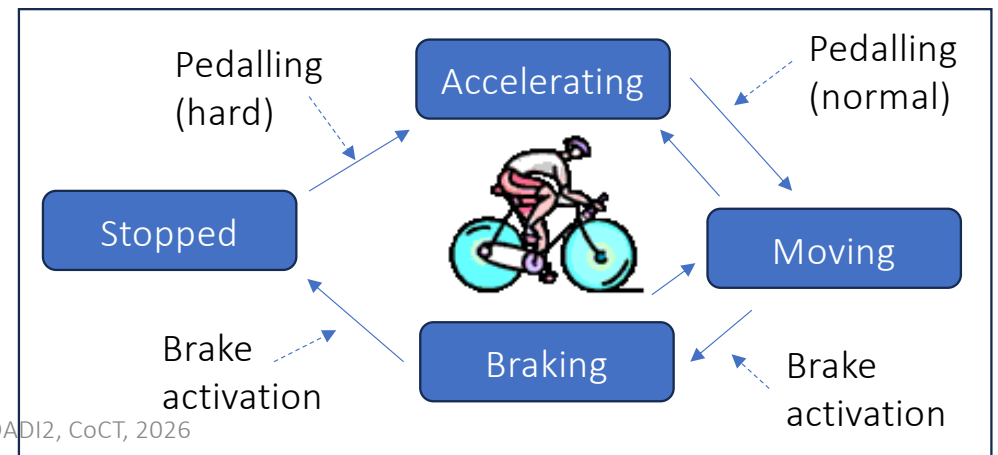
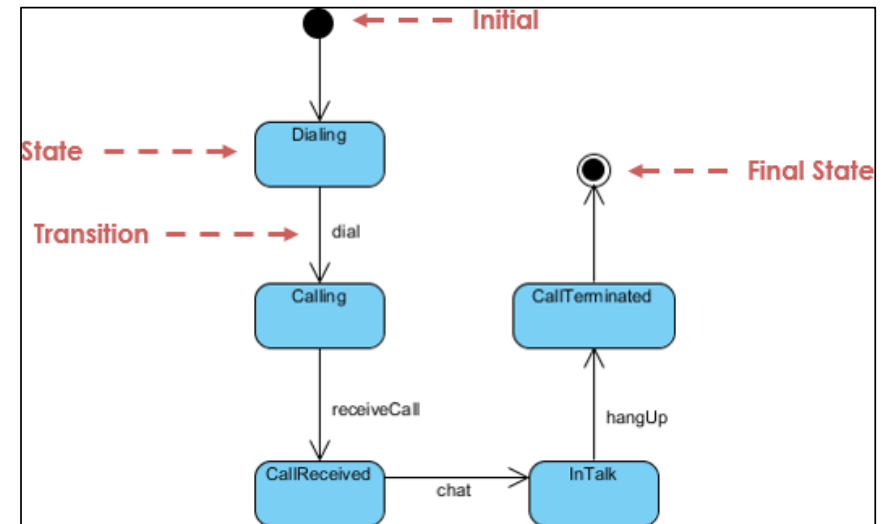
BREAK – PAUSE(FISK)

52



OBJECT STATES

- Some objects are merely placeholders for data (attributes)
- Others are complex and contain several states that influence how they work
- Transitions often happen due to external events, and require input from the outside world
- State diagrams are useful to document how the class works
 - Can be described at many levels
 - Best if states are also visible in the code



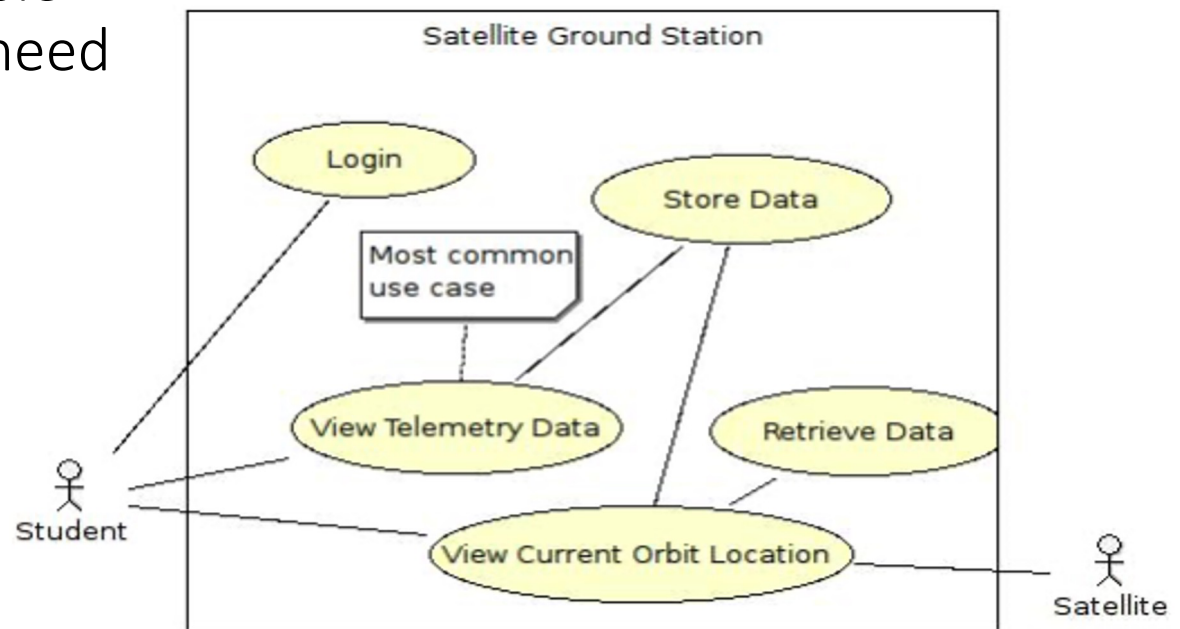
CLASSES – WHICH ONES DO WE HAVE/NEED?

- The core aspect is that we need to model the context our code will be working in
- Data modelling is a key aspect
 - What data is going to be used?
 - How is it going to be used?
 - When is it going to be exchanged, how and between whom?
- What concrete elements exists in our system?
 - What can we do with them and how?
 - What do we need from these elements?
 - What role do they have?

EXAMPLE – START WITH USE-CASE DIAGRAM

55

- Looking at a use-case diagram (usually) provides us with a quick overview and a very good idea of what classes we need to implement
- Here, in this simple example we see that we (at least) need five modules/classes
 - Login
 - Store data
 - Retrieve data
 - View telemetry data
 - View current orbit location



EXAMPLE – START WITH USE-CASE DIAGRAM

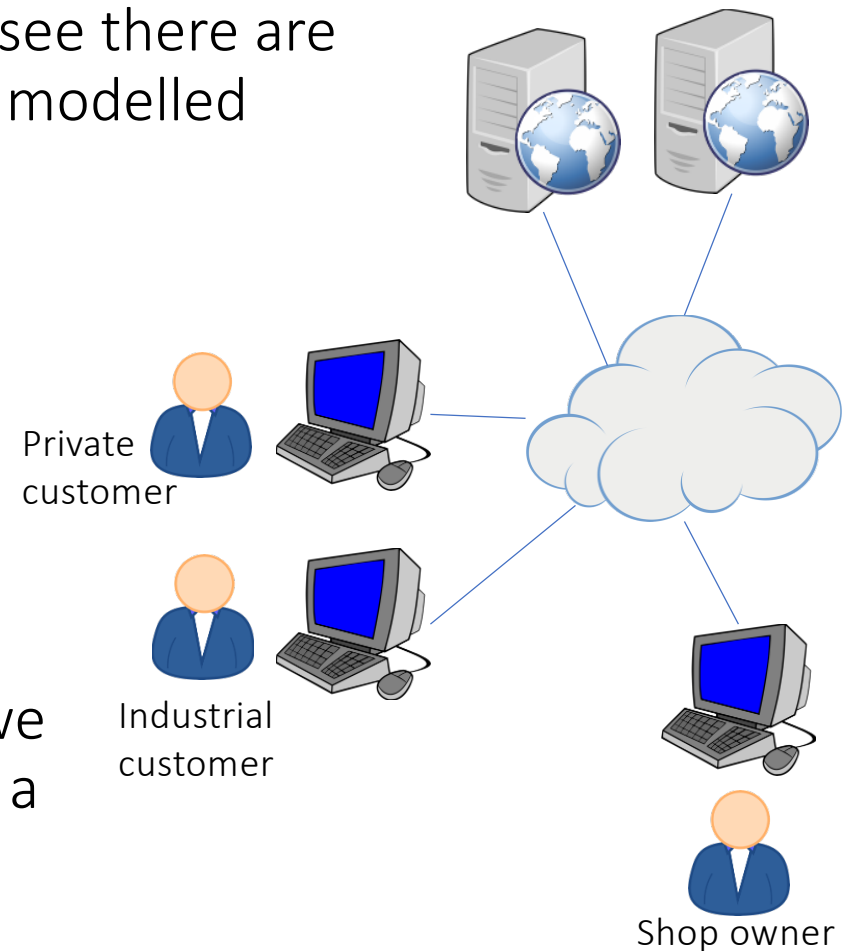
56

- If we look a bit closer at the ‘View telemetry data’ module we might find something along these lines
 - The user specifies a **set of data** (scientific data, satellite household data, alarms, etc.) that he/she wants to view from a **list**
 - Then he/she specifies a **start** and/or **end time**
 - The system retrieves the specified data from **persistent storage** and presents it via **GUI**
 - In case start or end times are left unspecified by the user, **default values** are substituted automatically
- **Nouns**: They represent potential class or attribute candidates

EXAMPLE – SYSTEM VIEW

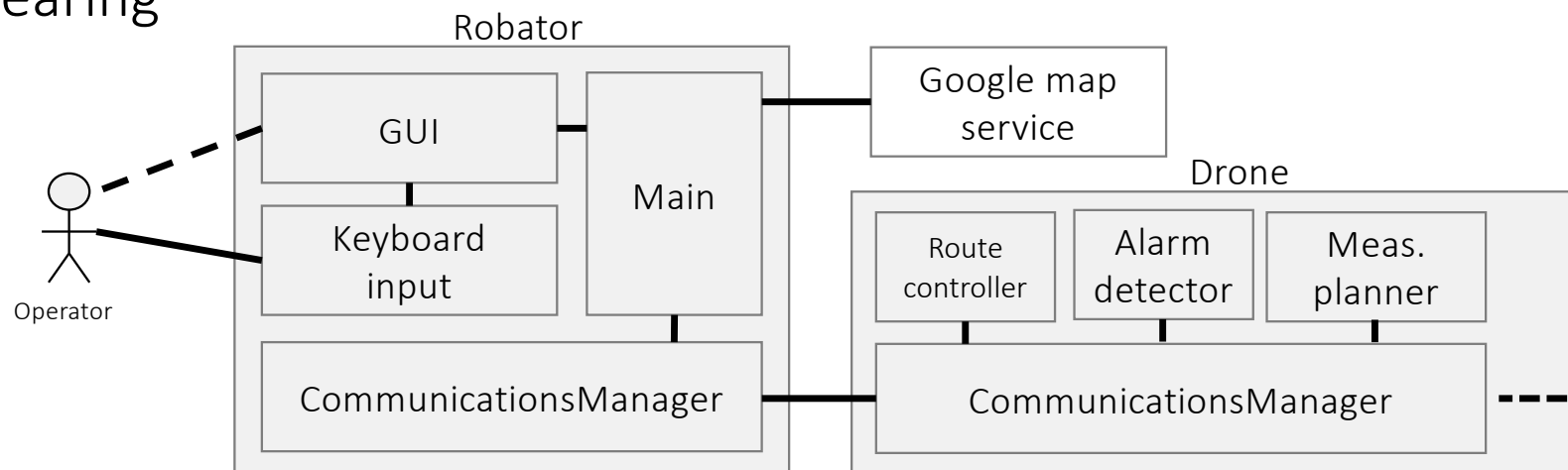
57

- For a given system drawing, we may easily see there are some obvious components – which can be modelled
 - Client, which may come in different flavours
 - For private customers
 - For Industrial customers
 - For shop owner
 - Server (we only need one as we just have different instances)
 - Items for sale
 - Money (to be exchanged)
- Basically, a starting point is given by what we can see – code is thereafter constructed in a 1:1 manner (or as close as possible)



EXAMPLE – MODULARIZED SYSTEM VIEW

- The next step is to look at modularization
- Based on the analysis leading to the use-case diagrams, and the general system view, it is possible to form a picture of a modularized system view
- As mentioned earlier, we now start to see module/class names appearing



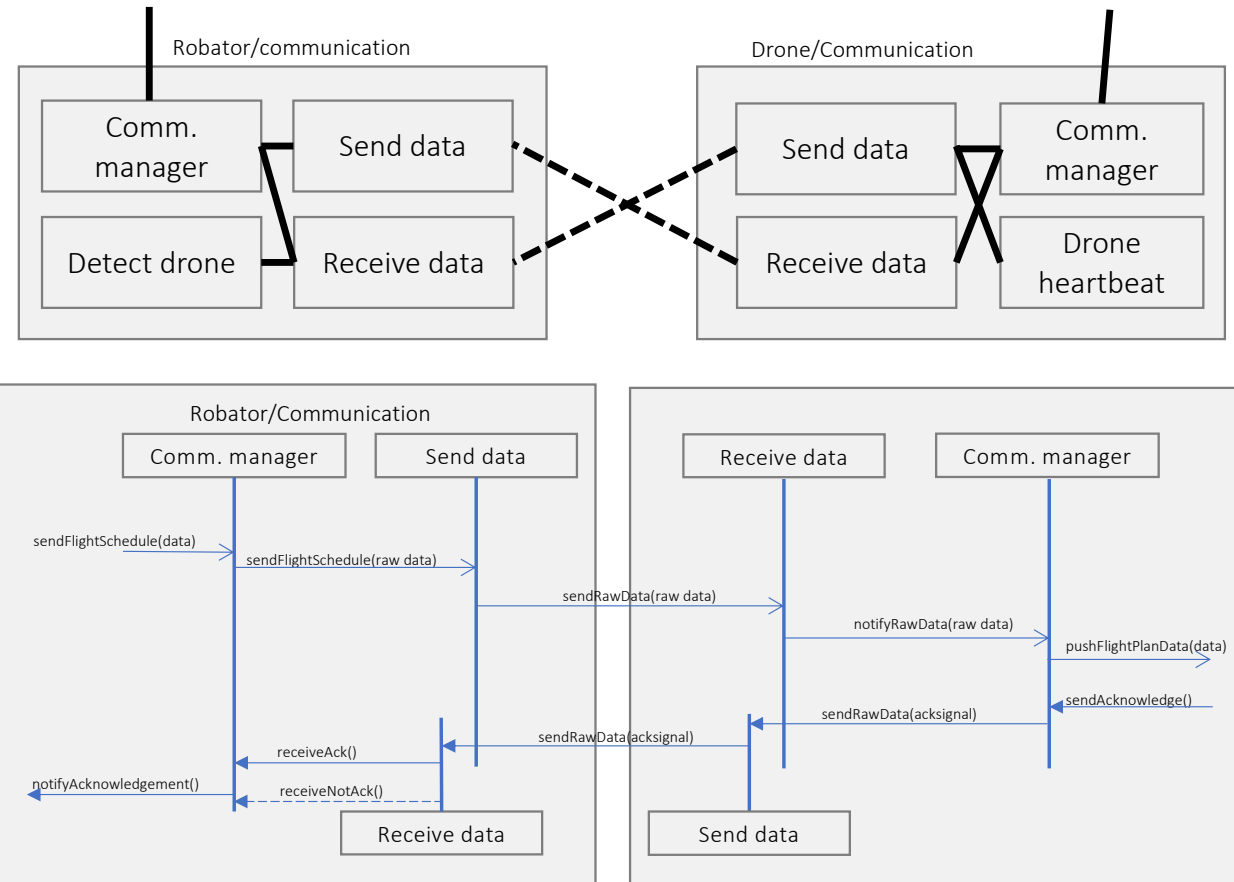
EXAMPLE – MODULARIZED SYSTEM VIEW

- One natural start is to simply state that our modules can be modelled as classes directly
 - **Robator** ,which has a **GUI**, **Main**, **Keyboard Input** and **Communication protocol**
 - **Drone**, which has a **Route Controller**, **Alarm detector**, **Measurement Planner** and a **Communication Protocol**
- If we do it right, all classes intuitively make sense
 - Main seems a bit generic and close to the start method – perhaps Core is better?
 - Keyboard input is also a bit strange – maybe just Keyboard as this is an object, which implicitly have input functionality
 - The critical part – no right or wrong answers, but some choices are more intuitive than others. An intuitive design gives us a high likelihood of writing **good** code
 - Maintainable, debuggable, understandable, extensible, etc.

EXAMPLE – WHAT METHODS TO USE?

- We need to understand how we want to use our objects and here designs, diagrams, subdivision, etc. give insight
- At first, naming method calls may not be obvious, but just the fact we know when and how to interact gives a strong indication of what methods we want

- Communication manager needs to send flight schedule using our module SendData (sends any generic data)
- **Why not add a method called `senFlightSchedule(..)`?**
- Due to the asynchronous approach, we need also something to receive and ack or nack
- **Why not add a `receiveAck()`?**
- We can also add a `receiveNotAck()` method that puts the Comm.Manager into a different state than `receiveAck()`





THINKING CHALLENGE

61

- Adding `senFlightSchedule(..)` would violate the overall design goal of keeping modules generic. Adding `sendFlightSchedule(...)` would
- Break generic design
 - Suddenly the module is no longer generic — it now “knows” about a *specific* type of data (flight schedules)
- Increase coupling
 - The communication logic becomes coupled to the domain concept *flight schedule*
Other modules using `SendData` (weather, maintenance, crew info, etc.) would not fit naturally
- Reduce reusability
 - A generic `SendData` module can be reused in many systems, which a `sendFlightSchedule()` method cannot
- Lead to method explosion
 - If we add `sendFlightSchedule()`, what next? `sendPassengerList()`, `sendGateChanges()`, `sendFuelStatus()`, etc.
 - This is brittle and grows uncontrollably
- Correct approach
 - Keep the interface generic



THINKING CHALLENGE

62

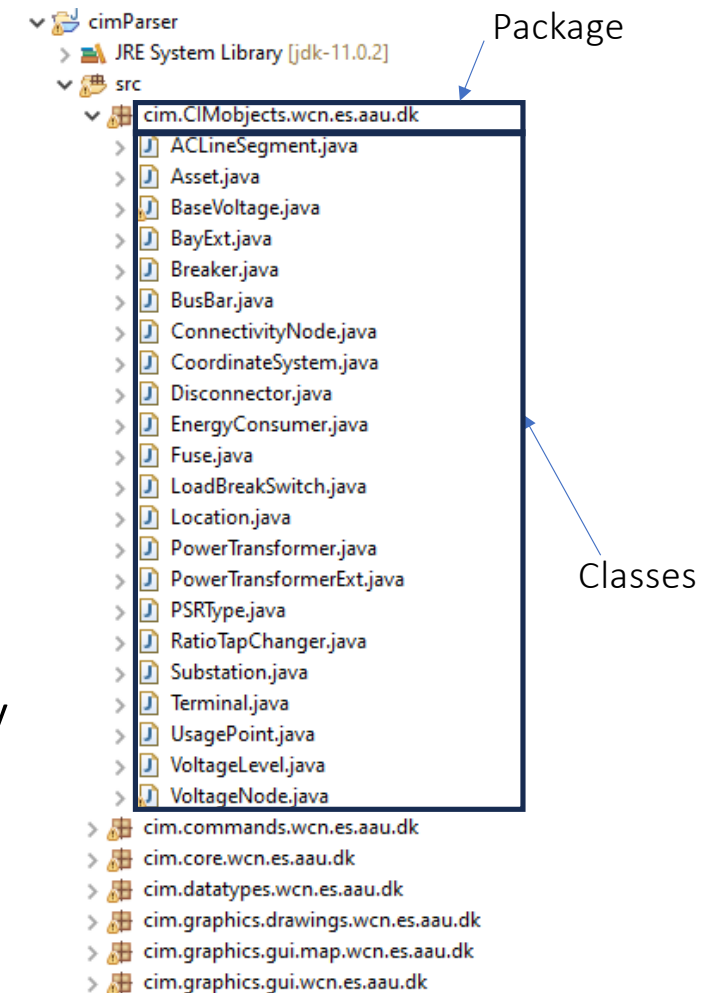
- Adding a `receiveAck()` would expose internal protocol details of the communication module. We should expose only one generic receive callback
- Leaks low-level protocol details into higher-level modules
 - The communication manager need not know exactly *how* acknowledgements look
- Tight coupling between sender and receiver implementation
 - If the protocol later changes (ACK becomes a structured object or includes delays, retries, timestamps, etc.), you would have to change the entire public API
- Leads to scattered state-handling
 - High-level components start reacting to ACK/NAK specifics rather than *communication outcomes*
- More methods = more states = more bugs
 - Each method represents a possible state transition and multiplying leads to fragile design
- Correct approach
 - Expose one generic method or event

ORGANIZING YOUR CODE

- Once we know which objects we wish to have, we should also have a structure to these
- Java offers packages (other languages have other similar concepts)

```
package cim.core.wcn.es.aau.dk;  
  
import cim.CIMObjects.wcn.es.aau.dk.Asset;  
...
```

- This is a good alternative to having all classes spread around in a single directory as this quickly becomes very messy



THE IMPORTANT STUFF

- Start with system behavior, not code
 - Use-case diagrams clarify actors, interactions, and goals before coding begins, ensuring system clarity
 - Understand **what the system must do** before thinking about **how** to implement it
- UML visualizes architecture
 - UML-diagrams structure team thinking to visualize architecture, class responsibilities, and communication pathways
- Meaningful classes and responsibilities
 - Classes should represent domain concepts with clear, non-overlapping responsibilities for maintainability
- Objects combine state and behavior
- Methods operate on an object's data, reflecting real-world behavior and encapsulation

- We all sometimes benefit from a different description or perspective, so consider checking out:
 - Dathan, Brahma, and Sarnath Ramnath. Object-Oriented Analysis, Design and Implementation: An Integrated Approach. 2nd ed. 2015, Springer Nature, 2015, <https://doi.org/10.1007/978-3-319-24280-4>.
- Specifically, look at the Introduction and Basics of Object-Oriented Programming chapters
- It should be available to you through AUB



ASSIGNMENTS (UML & USE-CASE)

1. Make a use case diagram for a small robot system that cleans garbage in a canteen. You don't need to have all complete
2. Identify main modules that can realize the use cases
3. Select one or a few modules and subdivide these modules
4. Describe how those would need to interact
5. Describe what entities are involved and for each entity, describe what we need to be able to do with them
6. Make (simple) code that in the same way as demonstrated in the lecture, demonstrates a selected part of the interactions you defined in step 4)



ASSIGNMENTS (CLASSES & OBJECTS)

67

7. Based on the below description, define a set of classes for things on a bicycle down to nuts and bolts. Define methods and attributes for each – you don't have to overdo this. Just so you have enough to roughly model a bicycle and be able to "use" each class in a limited way. I.e. use your senses to scope yourself
8. Implement the main classes from your initial design. Add some simple functionality to key method that simply prints its functionality to the screen, e.g. "changing gear", "wheel off"
9. Create a bicycle class, that initiates the previous defined classes. Also add attributes and methods to the bicycle as you would use a normal bicycle
10. Create a main() that initiates the bicycle and show that you can execute via code, the below description

With my Bicycle I can ride it forward by stepping on the pedals, and by changing gears I can go faster if I choose a higher gearing, but it also becomes harder to step on the pedals. For that reason, I start in first gear. I have front and backlights which switch on if it becomes dark. The wheels I can take off when I need to fix holes in the tire. When I brake, I can use both front and rear brakes.